Task Papers. 499–504.
[49] Nitesh Pradhan, Manasi Gyanchandani, Rajesh Wadhvani, et al. 2015. A Review on Text Similarity Technique used in IR and its Application. *International Journal of Computer Applications* 120, 9 (2015), 29–34.
[50] Yunjia Qi, Hao Peng, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. 2024. Adelie: Aligning large language models on information extraction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7371–7387.
[51] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
[52] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. 2011. Finding a" kneedle" in a haystack: Detecting knee points in system behavior. In *2011 31st international conference on distributed computing systems workshops*. IEEE, 166–171.
[53] Yuan Kui Shen and David R Karger. 2007. U-REST: an unsupervised record extraction system. In *Proceedings of the 16th international conference on World Wide Web*. 1347–1348.
[54] Damien Sileo, Wout Vossen, and Robbe Raymaekers. 2022. Zero-shot recommendation as language modeling. In *European conference on information retrieval*. Springer, 223–230.
[55] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, Arjun Radhakrishna, and Gust Verbruggen. 2025. Datavinci: Learning syntactic and semantic string repairs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
[56] Mukul Singh, Gust Verbruggen, Vu Le, and Sumit Gulwani. 2024. Tabularis Revilio: Converting Text to Tables. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4056–4060.
[57] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27 (2014).
[58] Chris van der Lee, Chris Emmery, Sander Wubben, and Emiel Krahmer. 2020. The CACAPO dataset: A multilingual, multi-domain dataset for neural pipeline and end-to-end data-to-text generation. In *Proceedings of the 13th International Conference on Natural Language Generation*. 68–79.
[59] Sam Wiseman, Stuart M Shieber, and Alexander M Rush. 2017. Challenges in data-to-document generation. *arXiv preprint arXiv:1707.08052* (2017).
[60] Sam Witteveen and Martin Andrews. 2019. Paraphrasing with Large Language Models. In *Proceedings of the 3rd Workshop on Neural Generation and Translation*. 215–220.
[61] Xueqing Wu, Jiacheng Zhang, and Hang Li. 2022. Text-to-Table: A New Way of Information Extraction. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2518–2533.
[62] Gongsheng Yuan, Jiaheng Lu, Zhengtong Yan, and Sai Wu. 2023. A survey on mapping semi-structured data and graph data to relational data. *Comput. Surveys* 55, 10 (2023), 1–38.
[63] Bowen Zhang and Harold Soh. 2024. Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 9820–9836.
[64] Jiahua Zhang, Meijuan Tan, Jing Zhang, Xiaolu Zhang, Jun Zhou, and Chenliang Li. 2025. Retrieval augmentation for text-to-table generation. *Information Processing & Management* 62, 4 (2025), 104135.
[65] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. [n. d.]. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.
[66] Zikang Zhang, Wangjie You, Tianci Wu, Xinrui Wang, Juntao Li, and Min Zhang. 2025. A survey of generative information extraction. In *Proceedings of the 31st International Conference on Computational Linguistics*. 4840–4870.
[67] Wang Zhao, Dongxiao Gu, Xuejie Yang, Meihuizi Jia, Changyong Liang, Xiaoyu Wang, and Oleg Zolotarev. 2024. MedT2T: An adaptive pointer constrain generating method for a new medical text-to-table task. *Future Generation Computer Systems* 161 (2024), 586–600.
[68] Shaowen Zhou, Bowen Yu, Aixin Sun, Cheng Long, Jingyang Li, Haiyang Yu, Jian Sun, and Yongbin Li. 2022. A survey on neural open information extraction: Current status and future directions. *arXiv preprint arXiv:2205.11725* (2022).
[69] Jun Zhu, Zaiqing Nie, Ji-Rong Wen, Bo Zhang, and Wei-Ying Ma. 2006. Simultaneous record detection and attribute labeling in web data extraction. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. 494–503.

A Appendix

A.1 LLM instructions

Here are the instructions with the complete schema for both datasets. Under the attribute recommendation setting, the attributes for validation and testing are excluded from the schema and hidden from the LLM.

Extraction Instruction - Incidents

Your task is to analyze the given text of Incidents and extract the relevant information according to the specified JSON structure below.
Provide the extracted information in this JSON structure:

```
{
"Accident": {
      "0":{
            "Accident type": [],
            "Accident date": [],
            "Accident address": [],
            "Number of rounds fired": [],
            "Accident number": [],
            "Personnel arrived time": [],
      }
},
"Victim": {
      "0": {
            "Victim number": [],
            "Victim status": [],
            "Victim gender": [],
            "Victim age": [],
            "Victim based": [],
            "Hospital name": [],
            "Victim name": [],
            "Victim race": [],
            "Victim occupation": [],
            "Victim vehicle": [],
      },
      "1":{...},
      ...
},
"Suspect": {
      "0":{
            "Suspect gender": [],
            "Suspect number": [],
            "Suspect status": [],
            "Suspect age": [],
            "Suspect name": [],
            "Suspect description": [],
            "Suspect weapon": [],
            "Suspect vehicle": [],
            "Suspect occupation": [],
            "Suspect based": [],
            "Suspect race": [],
            "Prison name": [],
      },
```

```
        "1":{...},
        ...
}
}
```

Explanations about the JSON structure:
- Output only the JSON data.
- There can be three types of entities mentioned in the texts: Accident, Victim, and Suspect. There is at most one Accident in the text. There can be multiple Victims or Suspects in the text.
- For each entity, there are multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text. You should consider every attribute listed in the above structure. If an attribute is not present or cannot be determined, do not include the attribute in the output.

Here are some examples:
{examples}
Here is the input text:
{text}
Here are some hints:
The above text is most likely to contain the attributes: $\{H^p\}$.
The above text is most likely to contain the following values: $\{V^p\}$.

Extraction Instruction - Weather

Your task is to analyze the given text of Incidents and extract the relevant information according to the specified JSON structure below.
Provide the extracted information in this JSON structure:

```
{
"Weather": {
        "0":{
                "Temperature": [],
                "Snow status": [],
                "Cloud type": [],
                "Sunset time": [],
                "Rain status": [],
                "Weather area": [],
                "Time": [],
                "Weather type": [],
                "Wind speed": [],
                "Weather compass direction": [],
                "Maximum temperature": [],
                "Weather frequency": [],
                "Wind status": [],
                "Minimum temperature": [],
                "Sunrise time": [],
                "Weather occurring chance": [],
                "Wind direction": [],
                "Location": [],
                "Cloud status": []
        },
        "1":{...},
        ...
        }
}
```

Explanations about the JSON structure:
- Output only the JSON data.
- There is one type of entity mentioned in the texts: Weather. There can be multiple Weather in the text.
- For each entity, there are multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text. You should consider every attribute listed in the above structure. If an attribute is not present or cannot be determined, do not include the attribute in the output.

Here are some examples:
{examples}
Here is the input text:
{text}
Here are some hints:
The above text is most likely to contain the attributes: $\{H^p\}$.
The above text is most likely to contain the following values: $\{V^p\}$.

Discovery Instruction - Incidents

Your task is to analyze the given text and discover new information mentioned in the text.
Provide the discovered information in this JSON structure:

```
{
"Accident": {
        "0":{
                "⟨ new attribute 1⟩": [],
                "⟨new attribute 2⟩": []
        }
},
"Victim": {
        "0":{
                "⟨new attribute 1⟩": [],
                "⟨new attribute 2⟩": []
        },
        "1":{...},
        ...
},
"Suspect": {
        "0":{

                "⟨new attribute 1⟩": [],
                "⟨new attribute 2⟩": []
        },
        "1":{...},
        ...
},
```

}

Explanations about the JSON structure:
- Output only the JSON data.
- {explanation}
- For each entity, there may be multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text.

Here are some examples:
{example text}
Explanations about the examples:
- Given an input text and the known information, you are encouraged to discover new information for each entity.
- The new information is qualified as a new attribute if and only if (1) it is relevant and important to the domain, (2) it is unique and not overlapped with the known information, and (3) it has proper granularity level compared with the known attributes, which means it is not too high-level or low-level concepts.
- The new attribute must have a meaningful name and actual values from the text.
- Output qualified new attributes as many as possible.

Here is the input text:
{text}
Here is the known information:
{extracted table}

Discovery Instruction - Weather

Your task is to analyze the given text and discover new information mentioned in the text.
Provide the discovered information in this JSON structure:

```
{
"Weather": {
      "0":{
            "⟨new attribute 1⟩": [],
            "⟨new attribute 2⟩": []
      },
      "1":{...},
      ...
      }
}
```

Explanations about the JSON structure:
- Output only the JSON data.
- explanation
- For each entity, there may be multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text.

Here are some examples:
{discovery example}

Explanations about the examples:
- Given an input text and the known information, you are encouraged to discover new information for each entity.
- The new information is qualified as a new attribute if and only if (1) it is relevant and important to the domain, (2) it is unique and not overlapped with the known information, and (3) it has a proper granularity level compared with the known attributes, which means it is not too high-level or low-level concepts.
- The new attribute must have a meaningful name and actual values from the text.
- Output qualified new attributes as many as possible.

Here is the input text:
{text}
Here is the known information:
{extracted table}

Schema Prefix - Incidents

Here is a schema designed to document and manage detailed information and facts about criminal incidents or accidents, particularly those involving violence, such as shootings or other crimes. The attributes are organized into three main entities: the accident itself, the victims, and the suspects. For each entity, there are multiple attributes. The attributes are unique and in consistent styles. Here are the attributes in this schema:

Schema Prefix - Weather

Here is a schema designed to document and manage detailed information and facts about weather conditions from news reports, particularly those about location, time, temperature, wind, cloud, rain, and snow. The attributes are organized into one main entity: the Weather itself, and there are multiple attributes. The attributes are unique and in consistent styles. Here are the attributes in this schema:

Judge Instruction in Hybrid Integration Strategy

Your task is to analyze the given schema and judge whether the given concepts are semantically similar enough to be merged as one attribute. Here is the schema:
{schema_text}
Here are the concepts to be judged:
{elements}
Explanation about the task:
- The schema includes several unique attributes, which show the semantic granularity of attributes.
- If the above concepts (1) are semantically equivalent, or (2) one is semantically contained by the others, or (3) they represent a finer granularity compared with the existing attributes, they need to be merged into one single

Algorithm 2 Pruned Bottom-up Search for P^*

Require: attribute proposals Z, known attributes S, threshold δ
Ensure: P^*

```
1: P* = ∅, Br← {}, len← 1                          ▷ Initialize dictionary.
2: for p ∈ Z ∪ S do Br[p] = Z ∪ S \ {p}             ▷ Start by len=1.
3: end for
4: while |Br|> 0∧len< |Z| + 1 do                    ▷ Outer loop.
5:     nextBr← {}
6:     for p ∈ Br, z′ ∈ Br[e] do                    ▷ Inner loop.
7:         valid ← True, activeBr ← Br[p]
8:         for z ∈ p do:                            ▷ Check all subset of length |p|
9:             p′ = p \ {z} ∪ {z′} ▷ Replace one element, |p′| = |e|
10:            if p′ ∈ Br then
11:                activeBr ← activeBr ∩ Br[p′]     ▷ Prune.
12:            else
13:                valid← False
14:                Break
15:            end if
16:        end for
17:        if valid = True then    ▷ All proper subset of p is in P*.
18:            p′ = p ∪ {z′}
19:            if vds(p′) < 1 + δ then
20:                P* ← P* ∪ {p′}
21:                nextBr[p′] ← activeBr
22:            end if
23:        end if
24:    end for
25:    len←len+1, Br← nextBr                        ▷ Search for the larger size
26: end while
```

attribute. Then, output "One".
- Else, they should be at least two unique attributes, output "Two".
- Output only "Two" or "One".

A.2 Vendi Score

Vendi Score is the effective rank of the normalized similarity matrix,

$$vds(q) = \exp(-\mathrm{tr}(\frac{\mathcal{K}_{q\times q}}{|q|}\log\frac{\mathcal{K}_{q\times q}}{|q|})), \tag{26}$$

where $\mathcal{K}_{q\times q}$ is the similarity matrix of size $|q|\times|q|$ and $\mathrm{tr}(\cdot)$ means trace. Vendi Score is maximized as $|q|$ if $\mathcal{K}(i,j)=0, \forall i,j\in q, i\neq j$, meaning every two elements are totally different, and it is minimized as 1 if $\mathcal{K}_{q\times q}$ contains all 1s. Since $q\subseteq Z\cup S=B$. We could precompute and cache $\mathcal{K}_{B\times B}$ and acquire submatix $\mathcal{K}_{q\times q}$ from it. Given $\mathcal{K}_{q\times q}$, the computational complexity of is in $O(|q|^3)$ [13]. In our experiment, we set $\mathcal{K}(a,b)=(\mathrm{chrf}\beta(a,b)+\mathrm{chrf}\beta(b,a))/2$, to reflect the pattern similarity as in Equation 12.

A.3 Supplementary Discussion for Hybrid Integration Strategy

A.3.1 Pruned bottom-up search for low-diversity set. As established by Equation 18, if a set is in P^*, then all of its subsets must also be in P^*. In other words, the monotonicity of $div(\cdot)$ means $div(p\cup\{z\})\geq div(p), \forall z\notin p$. Leveraging this property, our algorithm computes P^* in ascending order of their size, enabling efficient pruning during the process, as in Algorithm 2.

Specifically, in each iteration of the outer loop, the algorithm examines all qualified sets of size len + 1. A dictionary Br maps each qualified set p of size len to a set of candidate attributes $\mathrm{Br}[p]\subseteq Z\cup S$ that may be added to p to form a larger qualified set. Within the inner loop, the algorithm considers a candidate extension $p'=p\cup\{z'\}$ for some $z'\in\mathrm{Br}[p]$. It first performs a fast dictionary lookup to verify that replacing any element $z\in p$ with z' yields a known qualified set. If this check succeeds, indicating that all proper subsets of p' are already known to be qualified, the algorithm computes the Vendi Score $vds(p')$ to decide the final answer.

A.3.2 Complexity of diversity checking. Let $N=|Z\cup S|$ be the total number of candidate attributes, and let $L=\max_{p\in P^*}|p|$ denote the size of the largest low-diversity set returned by the algorithm. For $i\in\{1,\ldots,L\}$, define $M_i=|\{p\in P^*:|p|=i\}|$, with $M_1=N$. Algorithm 2 constructs P^* by attempting to extend each qualified set of size i with at most $N-i$ candidate attributes. The total number of examined sets is therefore bounded by

$$|P'|\leq\sum_{i=1}^{L}(N-i)M_i=N\sum_{i=1}^{L}M_i-\sum_{i=1}^{L}iM_i=N|P^*|-\sum_{p\in P^*}|p|.$$

Obviously, the number of evaluated sets scales linearly with the output size $|P^*|$, i.e., $|P'|=O(N|P^*|)$. Each candidate extension incurs $O(i)$ dictionary lookups, and a small fraction proceed to a Vendi Score computation, yielding an overall time complexity of $O\big(\sum_{i=1}^{L}(N-i)M_i\cdot(i+C_{vds})\big)$.

In practice, the pruning step within the inner loop further restricts the branch set $\mathrm{Br}[p]$: the number of viable extensions for a set p never exceeds that of any of its subsets, and $\mathrm{Br}[p]$ shrinks as p is extended. Hence, the factor $(N-i)$ can be tightened. Empirically, most candidate attributes are semantically distinct and do not form duplicate pairs with any other proposal; only a small fraction participate in true redundancies. Consequently, the factor is far smaller than N, and L remains modest (as detailed in the experimental section around Table 4).

A.3.3 Complexity of contextualized analysis. Suppose there are N' duplicates (i.e., elements that can potentially be eliminated by a *yes* response), and $|B|-N'$ non-duplicate elements. Then there must exist sets $q_1,q_2,\ldots,q_{|B|-N'}$ in P^* that together cover all elements, because every duplicate must co-occur with at least one non-duplicate in some set belonging to P^*. Consequently, for the set cover instance defined by the collection P^* and the ground set B, the optimal solution size is strictly less than $|B|-N'$. A greedy set cover algorithm achieves an approximation ratio of $(\ln|B|+1)$ [10]. Therefore, the number of LLM calls incurred by Q is at most $(\ln|B|+1)(|B|-N')$. Line 10 further eliminates low-diversity sets that contain only a single element, which do not submit for LLM judgment.

Given the same B and $|P^*|$, in the worst case, the algorithm examines every set in P^* and receives *no* for all queries, incurring $|P^*|$ LLM calls without performing any deduplication. Such a situation is rare because each set in P^* is already known to be of very low diversity, meaning it plausibly contains duplicates. The analysis

above indicates that a greater number of duplicates allows an outer iteration to complete more quickly, which submits each element at least once for duplication. We prioritize preserving the breadth of the search because the ultimate effect of deduplication hinges on whether each duplicate element is eliminated. The greedy set cover actually processes larger sets earlier, which helps the algorithm eliminate confirmed duplicates with few queries and shrinks the search space for subsequent iterations.

A.3.4 Discussion of Hybrid Integration Strategy. Resolving duplicate attribute proposals is essential, but this step lacks ground truth and thus cannot be formulated as a rigorous optimization problem. The reason is that semantic equivalence between attributes is inherently subjective and highly dependent on the context. As with any recommendation system, the user's ultimate preference may deviate from what the history infers. For instance, given the earlier example of "Medical center name" versus "Hospital name," it is logical for a method to infer from the existing schema that the user likely intends to merge them. Yet a user might insist on retaining both to preserve fine-grained distinctions, much like a consumer abruptly deviating from past behavior to purchase an entirely different product.

Given this inherent uncertainty, our Hybrid Integration Strategy is pragmatic rather than exhaustive: we defer to the diversity metric and LLM's judgment and perform moderate processing to consolidate obvious duplicates, without pursuing a perfect formulation or solution. Our experiments further support this pragmatic stance (see Section 6.3(2)), which shows that pruning the most conspicuous redundancies already yields substantial gains.

A.4 Generating Attribute Proposal Definitions

Since each attribute proposal varies in occurrence frequency, number of mentions, and mention length across source texts, directly gathered raw contexts can be highly inconsistent in length and style. This variability introduces bias to the LLM and could result in overly long input that hinders reliable reasoning. To address this, we first normalize each proposal's context into a concise definition using the LLM itself.

This step produces a standardized context(z) for every attribute proposal$z$ that needs to be judged by the LLM, ensuring consistent style and length and improving fairness. Let $D^{u+}_{i_1}, D^{u+}_{i_2}, \ldots$ denote the pseudo-tables containing the attribute proposal z. The LLM instruction for generating a standard context is shown below.

Generating context(z) for attribute proposal z.

Your task is to summarize the context of an attribute and write a definition for it.
Provide the definition in this JSON structure:
{ "<name>": "<definition>", }
Explanations about the JSON structure:
- Output only the JSON data, where the key is the name of the attribute and the value is its definition.
- { task description }
- The name should be concise and in the same style with the examples.
- The name should be of proper granularity. Note that if it is too general, it cannot accurately describe the range of values; if it is too specific, it cannot cover all the values.
- The definition should precisely describe the meaning and characteristics of the attribute, enabling the reliable extraction of the attribute from other texts.
- The definition should be less than 50 words.
{ example definition }
Here is the context of the attribute to be defined:
In the text $W^u_{i_1}$, the z value are: { values in $D^{u+}_{i_1}$ }
In the text $W^u_{i_2}$, the z value are: { values in $D^{u+}_{i_2}$ }
...

Here the "task description" is a brief description of the dataset, and "example definitions" is the definition for known attributes in the schema, if there are any.

A.5 Discussion of existing datasets

Table 5 summarizes all open-source datasets used in previous table extraction from text methods in contrast to ours.

A.6 Data Annotation

We first deduplicate the original texts and remove corrupted texts. Three annotators (A, B, and C) participate in creating the table annotations. For each dataset, the annotation proceeds in three steps. First, the three annotators read the original CACAPO documentation and discussed the original attribute set to reach a consensus on the overall schema. Second, we sample 100 texts, which are independently annotated by A, B, and C. During annotation, each annotator is provided with the original attribute-value pairs and the complete table extraction schema. They are asked to fill in missing values, correct errors in the original annotations, and organize the correct values into multiple records (rows) to form a complete table. Disagreements are discussed until consensus is reached. The remaining texts are then split into two parts and assigned to B and C, respectively. From each part, we sample another 100 texts for A to annotate independently. The pairwise agreement between A and B is 88%, and between A and C is 82%.

A.7 Discussion on Structured-F1

We discuss the computational complexity of structured-F1. Assume D has n rows and m columns while $\hat{D}$ has $\hat{n}$ rows and $\hat{m}$ columns. Let c denote the amortized cost to calculate the similarity between two cell contents. To compute DF1 for some record pair, it requires $O(m\hat{m}c)$ to get the similarity matrix between header names, as in Figure 3(1), and $O((m+\hat{m})c)$ to average the similarity of matched values. We need to compute DF1 for all $n\hat{n}$ record pairs, the f_k similarity matrix could be reused, so the overall complexity for structured-F1 is $C1 = O(m\hat{m}c + n\hat{n}(m+\hat{m})c)$.

In contrast, if the table is flattened, there are mn and $\hat{m}\hat{n}$ triples. So, the complexity of comparing triples is $C2 = O(mn\hat{m}\hat{n}c)$.

For a reasonable system, the size of D resembles that of $\hat{D}$, thus $m \approx \hat{m}$ and $n \approx \hat{n}$, so $C1 = O(m^2c + 2n^2mc)$, $C2 = O(m^2n^2c)$. When $n^2 > 1$ and $m > 3$, $C1 \leq C2$ and structured-F1 is more efficient than triple F1; in other cases, $C1 \approx C2$. In practice, the m and n

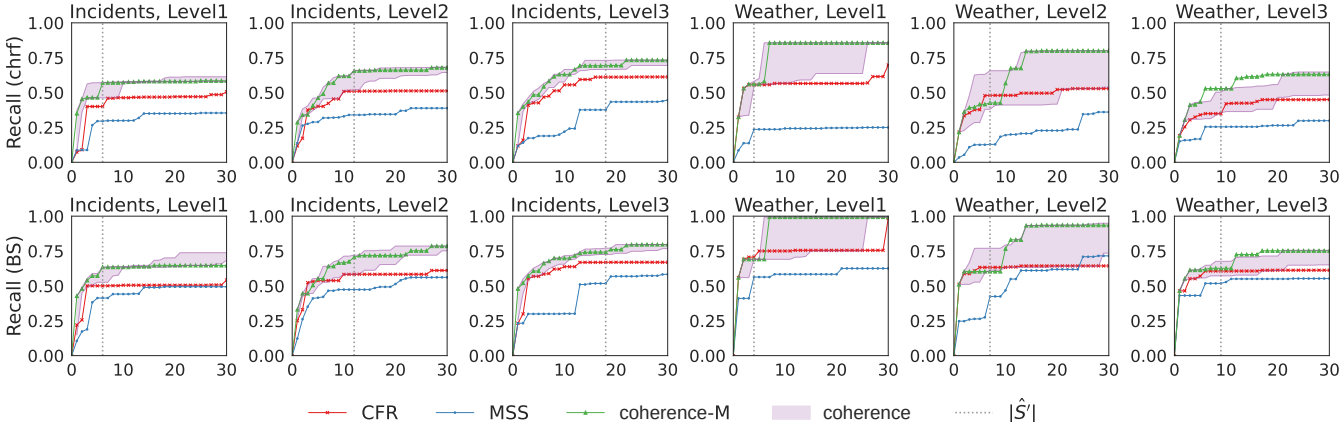


Figure 6: Recommendation recall with Llama.

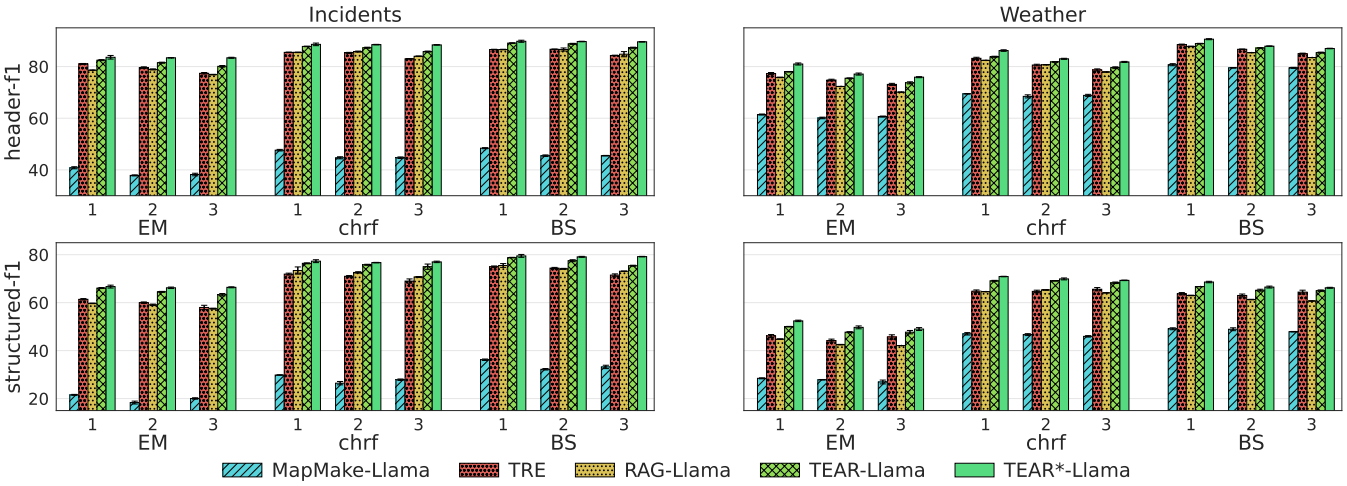


Figure 7: Table extraction with text-driven attributes, Llama backbone.

are usually small integers, so it is fair to say our metric does not increase the evaluation complexity.

A.8 Other Results

Figure 6 and Figure 7 are the results with Llama-backbone.

Table 5: Comparison of datasets. "Multiple" indicates whether each table contains multiple records or a single record.

Dataset	Input texts	Output table	Comments	Multiple
E2E [40, 41, 61]	human-written description	synthetically generated restaurant attributes	table description	No
RotoWire [59, 61]	human-written game summary	NBA box score	table description	Yes
WikiTableText [2, 61]	human-written table description	Wikipedia web tables	table description	No
WikiBio [25, 61]	filtered Wikipedia biographies	Wikipedia web tables	specialized documents	No
CPL [24]	Chinese private lending judgments	local tabular views of a KG	specialized documents	Yes
Incidents (Ours)	gun-violence news report [58]	manually-annotated tables about the victim, suspect and accident	naturally occurring text	Yes
Weather (Ours)	weather forecasts in news [58]	manually-annotated tables about weather details	naturally occurring text	Yes